

E-commerce Platform



DevOps Implementation - Microservices Architecture

Microservices

CI/CD

GitOps

Argo CD

Kubernetes

نظرة عامة



منصة تجارة إلكترونية حديثة تعتمد على معمارية Microservices، مُدارة بالكامل عبر GitOps باستخدام Argo CD على Kubernetes. يوضح المشروع أفضل ممارسات DevOps في بيئة سحابية قابلة للتوسع.

المزايا الرئيسية: ✓

- معمارية Microservices قابلة للتوسع
- نشر تلقائي باستخدام GitOps (Argo CD)
- CI/CD Pipeline مع GitHub Actions
- Ingress Controller لإدارة حركة المرور
- Health Monitoring لجميع الخدمات

المعمارية التقنية



Ingress Controller



Payment API

Python

Port: 3002

NodePort: 32132

User API

Node.js

Port: 3001

NodePort: 32047

Frontend

React

Port: 3000

NodePort: 31322

الخدمات الرئيسية (Microservices)

الوصف	NodePort	المنفذ	Cluster IP	التقنية	الخدمة
واجهة المستخدم الرئيسية	31322	3000	10.96.58.68	React + Nginx	Shopping Cart Frontend
إدارة المستخدمين والجلسات	32047	3001	10.99.26.40	Node.js + JSON	User Management API
معالجة عمليات الدفع	32132	3002	10.104.12.154	Python + Flask	Payment Processing API

التقنيات المستخدمة

CI/CD & GitOps 

Infrastructure 

Argo CD: GitOps Deployment •

GitHub Actions: CI/CD •
Pipeline

Helm: Package Management •

Git: Version Control •

Kubernetes: Container •
Orchestration

Minikube: Local •
Development

Docker: Containerization •

Nginx Ingress: Load •
Balancing

LoadBalancer Services: •
External Access

خطوات التثبيت والإعداد

Minikube Cluster إعداد

```
# تشغيل Minikube
minikube start --cpus=2 --memory=3072

# تفعيل Ingress
minikube addons enable ingress

# التحقق من الحالة
kubectl cluster-info
```

Argo CD تثبيت

```
# إنشاء namespace
kubectl create namespace argocd

# تثبيت Argo CD
kubectl apply -n argocd -f \
https://raw.githubusercontent.com/argoproj/argo-
```

```
cd/stable/manifests/install.yaml
```

```
# الحصول على كلمة المرور  
kubectl -n argocd get secret argocd-initial-admin-secret \  
-o jsonpath="{.data.password}" | base64 -d
```

إنشاء Application 3

```
# تسجيل الدخول  
argocd login localhost:8080 --username admin --password <PASSWORD> --  
insecure  
  
# إنشاء التطبيق  
argocd app create ecommerce-platform \  
--repo https://github.com/mohammed-makram/ecommerce-platform.git \  
--path templates \  
--dest-server https://kubernetes.default.svc \  
--dest-namespace default  
  
# المزامنة  
argocd app sync ecommerce-platform
```

إعداد DNS المحلي 4

```
# إضافة إلى /etc/hosts  
echo "$(minikube ip) ecommerce.example.com" | sudo tee -a /etc/hosts
```

روابط الوصول للخدمات

الوصف	الرابط	الخدمة
واجهة المستخدم	/http://ecommerce.example.com	Frontend

الوصف	الرابط	الخدمة
الرئيسية		
API إدارة المستخدمين	http://ecommerce.example.com/user-management-api/health	User API
API معالجة الدفع	http://ecommerce.example.com/payment-processing-api/health	Payment API
لوحة تحكم Argo CD	https://localhost:8080	Argo CD UI

CI/CD Pipeline

يتم تشغيل Pipeline تلقائياً عند:

- Push إلى branch master
- Merge Pull Request

GitHub Secrets المطلوبة

الوصف	القيمة المتوقعة	اسم Secret
عنوان Argo CD Server	localhost:8080	ARGOCD_SERVER
اسم المستخدم	admin	ARGOCD_USERNAME
كلمة المرور	من Argo CD Secret	ARGOCD_PASSWORD

أوامر المزامنة اليدوية

```
# مزامنة التطبيق
argocd app sync ecommerce-platform

# فرض المزامنة
argocd app sync ecommerce-platform --force

# متابعة الحالة
argocd app wait ecommerce-platform --health
```

التحقق والاختبار

فحص Services

```
kubectl get svc
kubectl describe svc
<service-name>
```

فحص Pods

```
kubectl get pods
kubectl describe pod <pod-
name>
```

Health Check Endpoints

```
# اختبار Frontend
curl http://ecommerce.example.com/

# اختبار User API
curl http://ecommerce.example.com/user-management-api/health

# اختبار Payment API
curl http://ecommerce.example.com/payment-processing-api/health
```

المشاكل الشائعة والحلول: 

مشكلة: Pods في حالة Pending

```
kubectl describe pod <pod-name>
kubectl top nodes
```

مشكلة: Ingress لا يعمل

```
kubectl get pods -n ingress-nginx
kubectl describe ingress ecommerce-ingress
```

مشكلة: Argo CD Sync فاشل

```
argocd app get ecommerce-platform
argocd app sync ecommerce-platform --force
```

عرض Logs

```
kubectl logs -f deployment/shopping-cart-frontend
kubectl logs -f deployment/user-management-api
kubectl logs -f deployment/payment-processing-api
```

```
ecommerce-platform/
├── frontend/ # React shopping cart UI
├── backend-py/ # Python payment service
├── backend-json/ # Node.js user service
├── .github/
│   └── workflows/
│       └── deploy-argo.yml # CI/CD pipeline
├── templates/ # Kubernetes manifests
│   ├── deployments.yaml
│   ├── services.yaml
│   └── ingress.yaml
├── Chart.yaml # Helm chart
├── values.yaml # Configuration
└── output.yaml # Rendered manifests
```

ملاحظات مهمة

⚠️ للبيئة الإنتاجية، يُنصح بإضافة:

- تفعيل TLS/SSL Certificates ✓
- استخدام Secrets Management (Vault) ✓
- إضافة Monitoring (Prometheus + Grafana) ✓
- تطبيق Quotas والـ Resource Limits ✓
- إعداد Backup و Disaster Recovery ✓
- استخدام Production Ingress Controller ✓

 معلومات إضافية:

- جميع الخدمات توفر health endpoint/ للمراقبة
- Frontend يحصل على البيانات من Backend APIs

- يجب تشغيل جميع الخدمات قبل الوصول للـ Frontend
- Minikube IP قد يتغير، تأكد من تحديث etc/hosts/

مصادر إضافية

Links

- kubernetes.io/docs
- argo-cd.readthedocs.io
- minikube.sigs.k8s.io
- docs.github.com/actions

Documentation

- Kubernetes Docs
- Argo CD Documentation
- Minikube Guide
- GitHub Actions